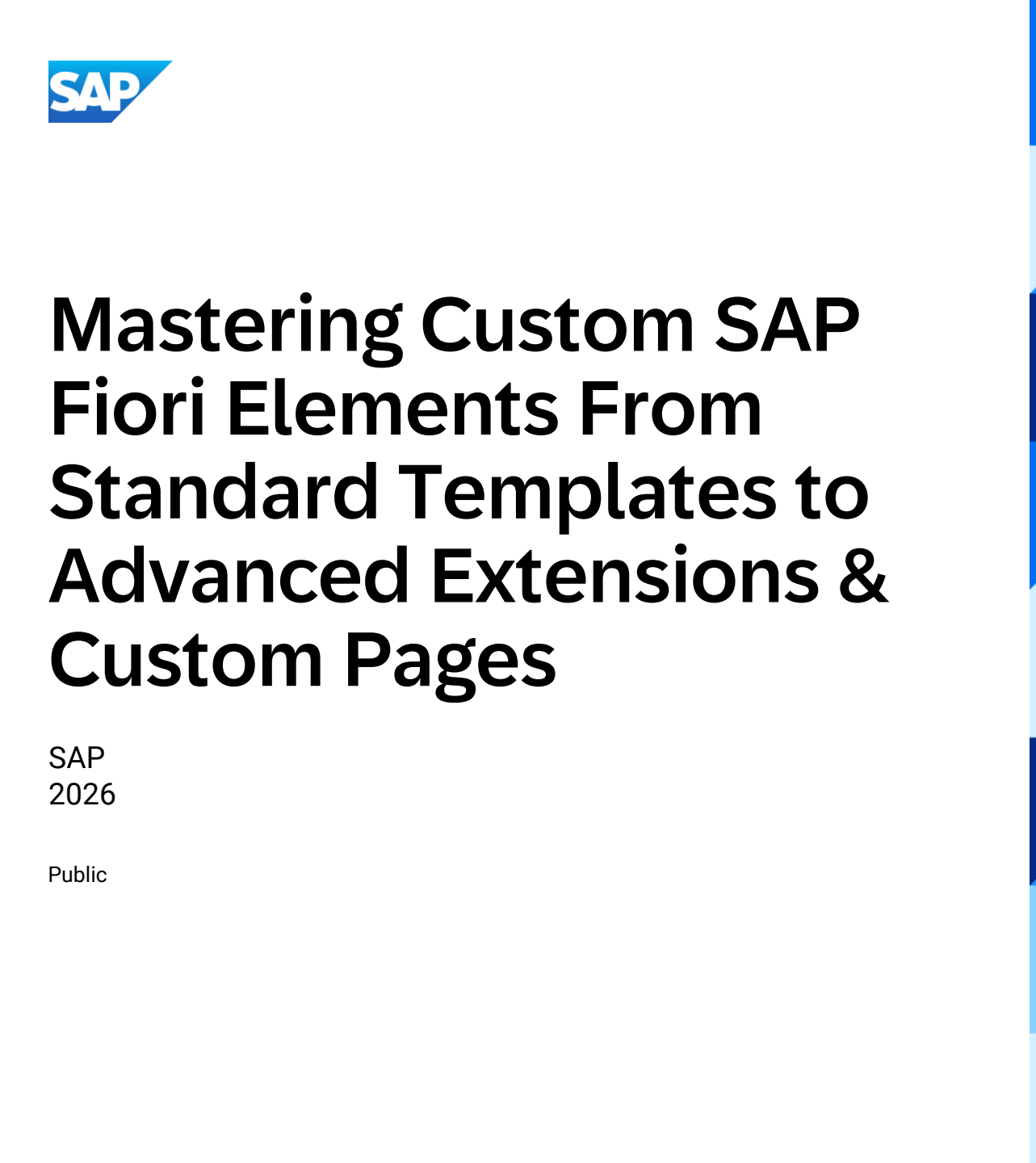


Mastering Custom SAP Fiori Elements From Standard Templates to Advanced Extensions & Custom Pages

SAP
2026

Public

Disclaimer

The information in this presentation is confidential and proprietary to SAP and may not be disclosed without the permission of SAP. Except for your obligation to protect confidential information, this presentation is not subject to your license agreement or any other service or subscription agreement with SAP. SAP has no obligation to pursue any course of business outlined in this presentation or any related document, or to develop or release any functionality mentioned therein.

This presentation, or any related document and SAP's strategy and possible future developments, products and or platforms directions and functionality are all subject to change and may be changed by SAP at any time for any reason without notice. The information in this presentation is not a commitment, promise or legal obligation to deliver any material, code or functionality. This presentation is provided without a warranty of any kind, either express or implied, including but not limited to, the implied warranties of merchantability, fitness for a particular purpose, or non-infringement. This presentation is for informational purposes and may not be incorporated into a contract. SAP assumes no responsibility for errors or omissions in this presentation, except if such damages were caused by SAP’s intentional or gross negligence.

All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.

Live Session Guidelines



The slides will be shared in PDF format after the live session.

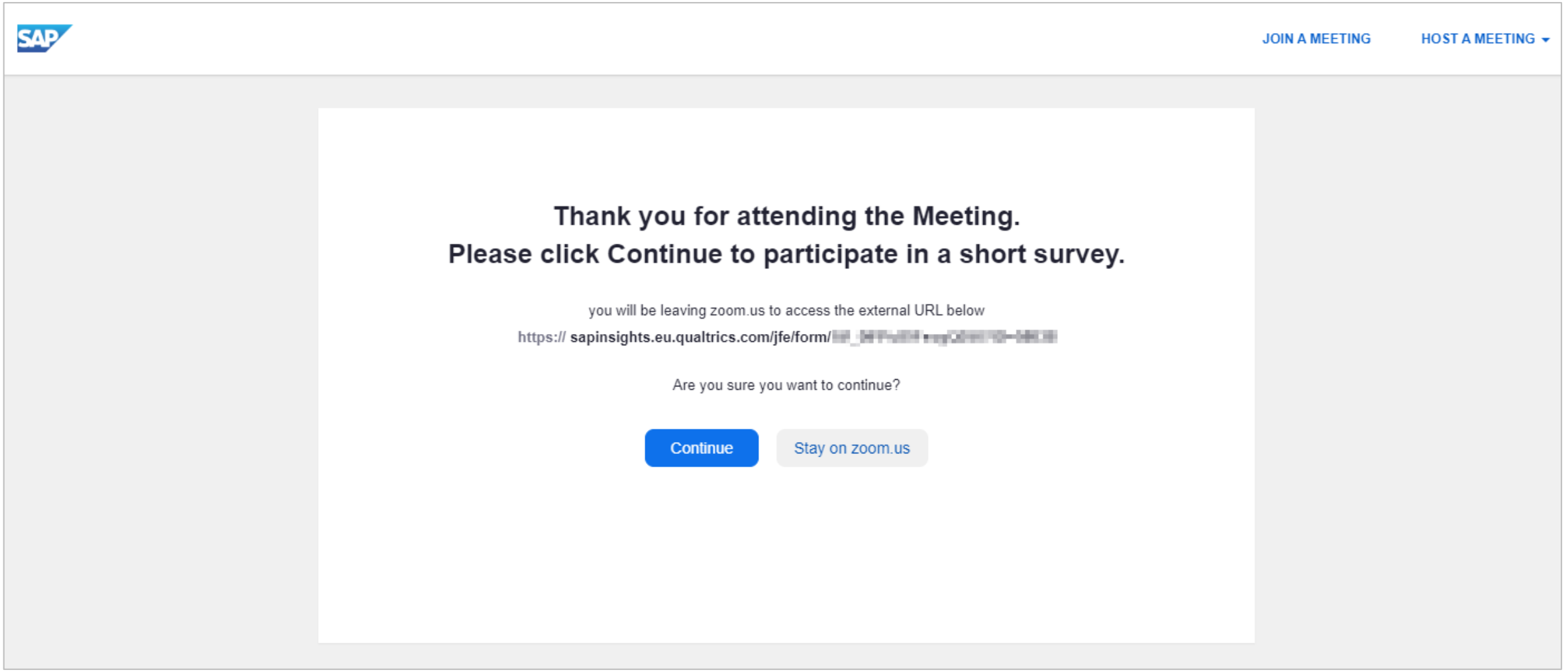


The live session is not being recorded.



Use the chat for questions during the session.

Please complete the survey when leaving the live session



Agenda


Foundations
The Extension Spectrum
Extension Techniques
Adaptations
Building for the Long Term

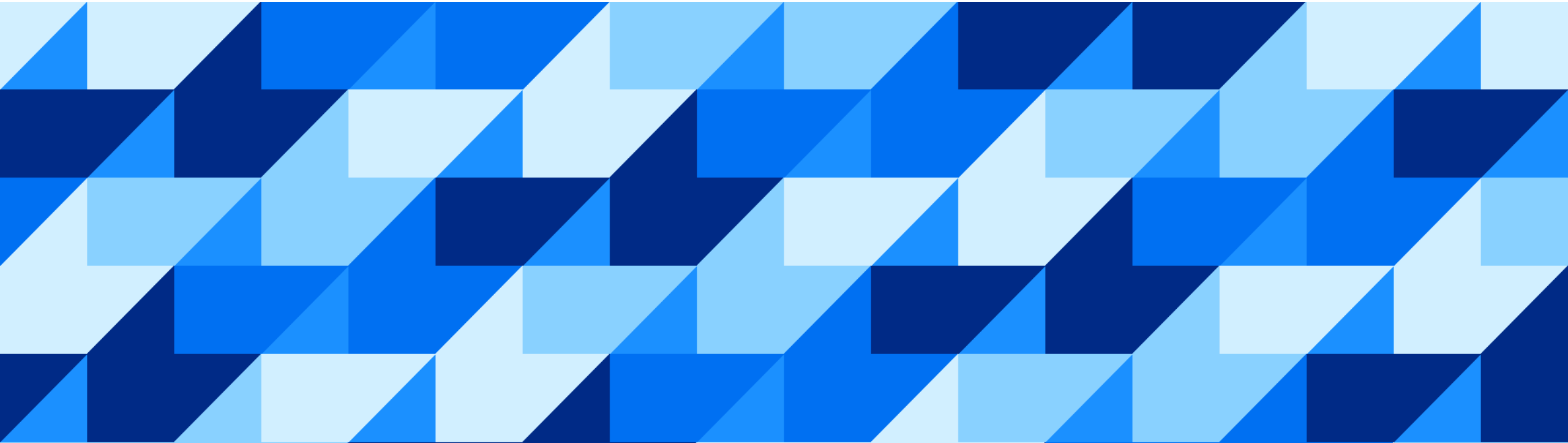
The Promise of Fiori Elements

Annotations + manifest = full application. No view, no controller, no JavaScript.

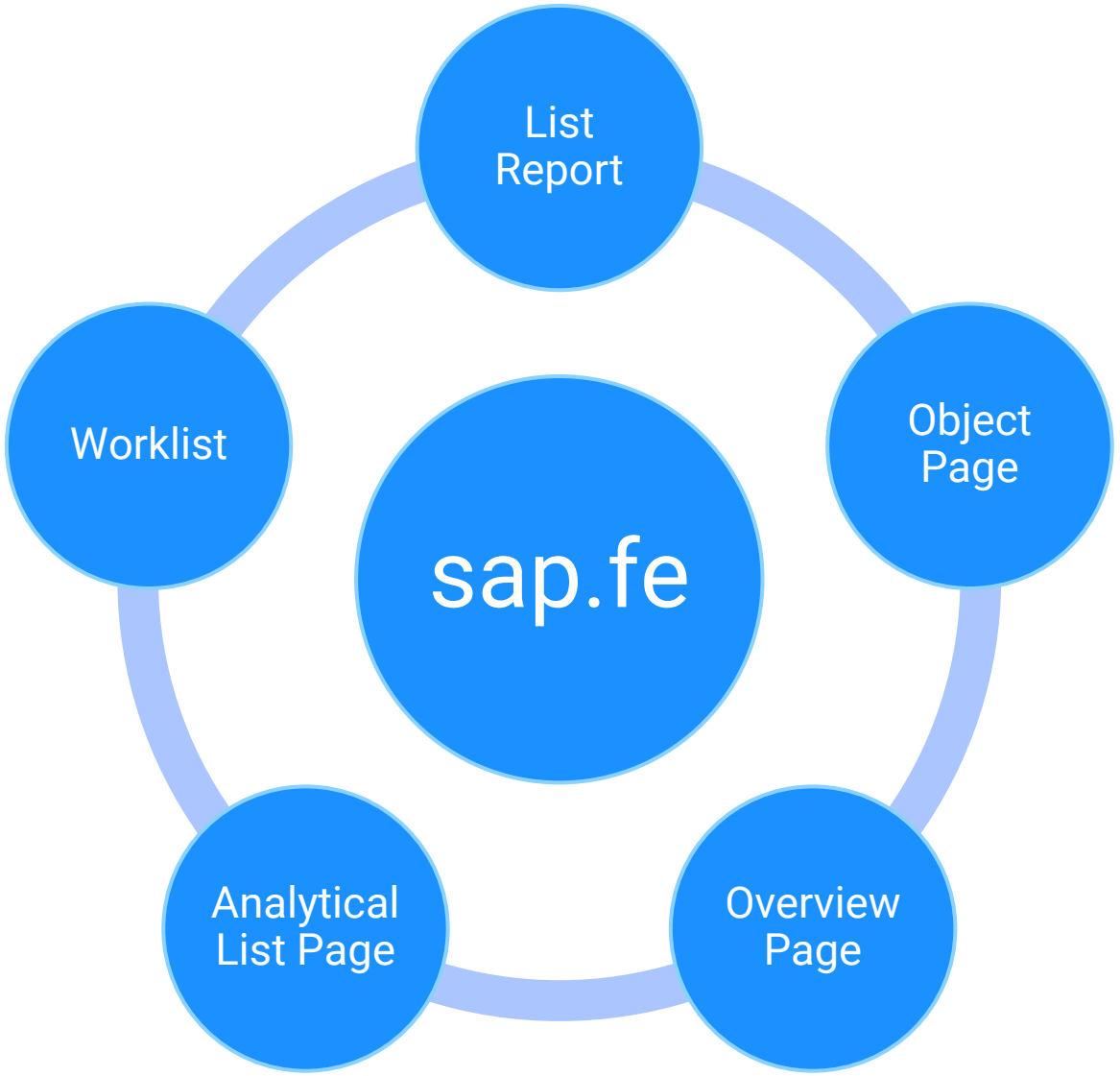
Trap that most teams fall into

The moment a real requirement nudges past the standard floorplan, teams either fight the framework or abandon it for freestyle. Both reactions are expensive.

Foundations



The Fiori Elements Landscape in 2025-2026



Vocabularies – the Language of Fiori Elements

Annotation = metadata attached to an OData element, following a published vocabulary

Vocabularies are SAP- or OASIS-owned namespaces with defined terms

Framework reads them at runtime → renders accordingly

UI	Common	Communication
• Layout	• Meaning	• Interactive Contact Data

UI Vocabulary – the Layout Terms

`UI.LineItem`

`UI.HeaderInfo`

`UI.HeaderFacets`

`UI.FieldGroup`

`UI.SelectionFields`

`UI.Identification`

`UI.DataPoint`

`UI.Chart`

`UI.PresentationVariant` / `UI.SelectionVariant`

Common Vocabulary

`Common.Label`

`Common.Text` / `Common.TextArrangement`

`Common.ValueList`

`Common.SemanticObject`

`Common.IsCurrency` / `Common.IsUnit`

`Common.DraftRoot` / `Common.DraftNode`

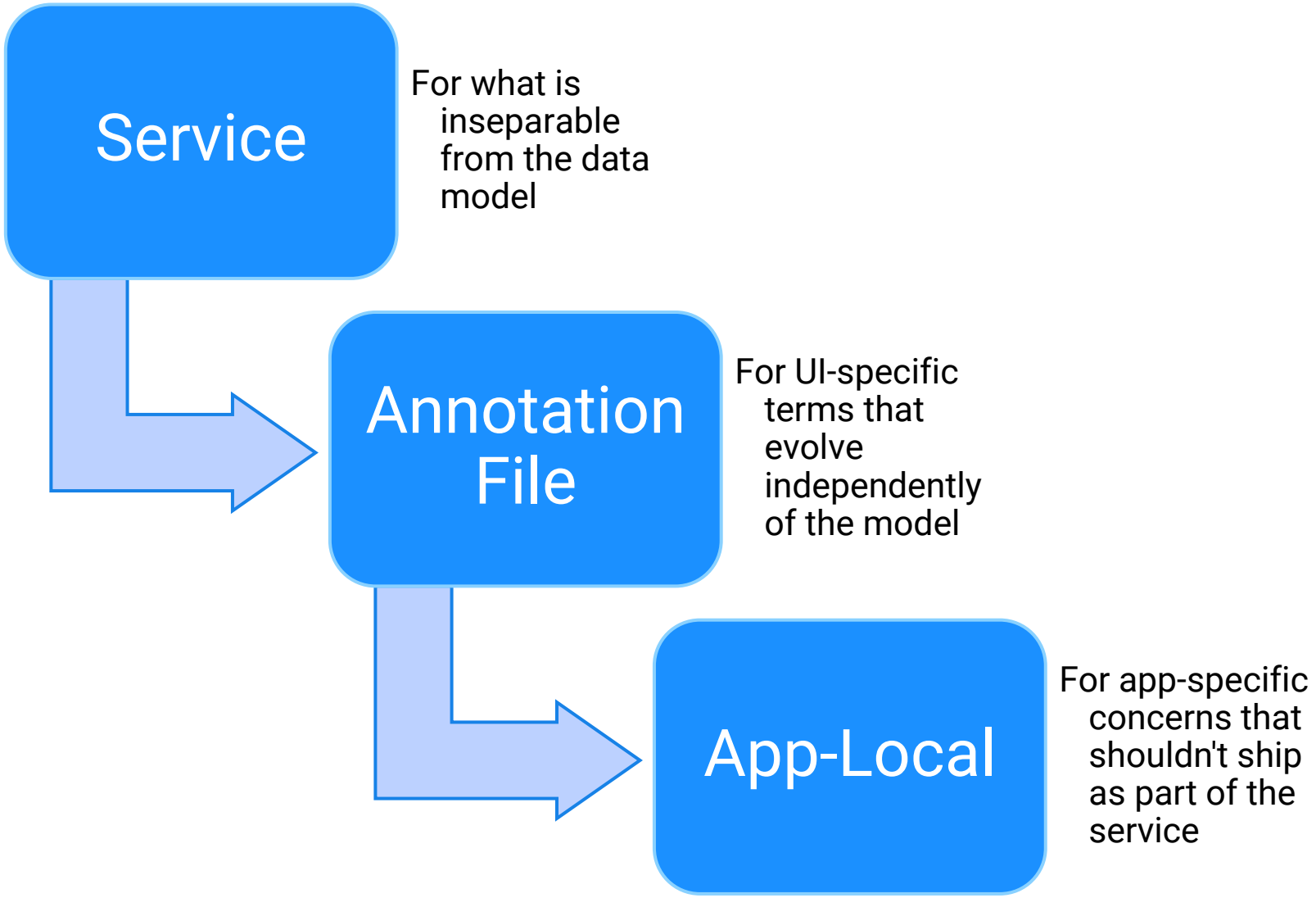
Communication Vocabulary

`Communication.Contact`

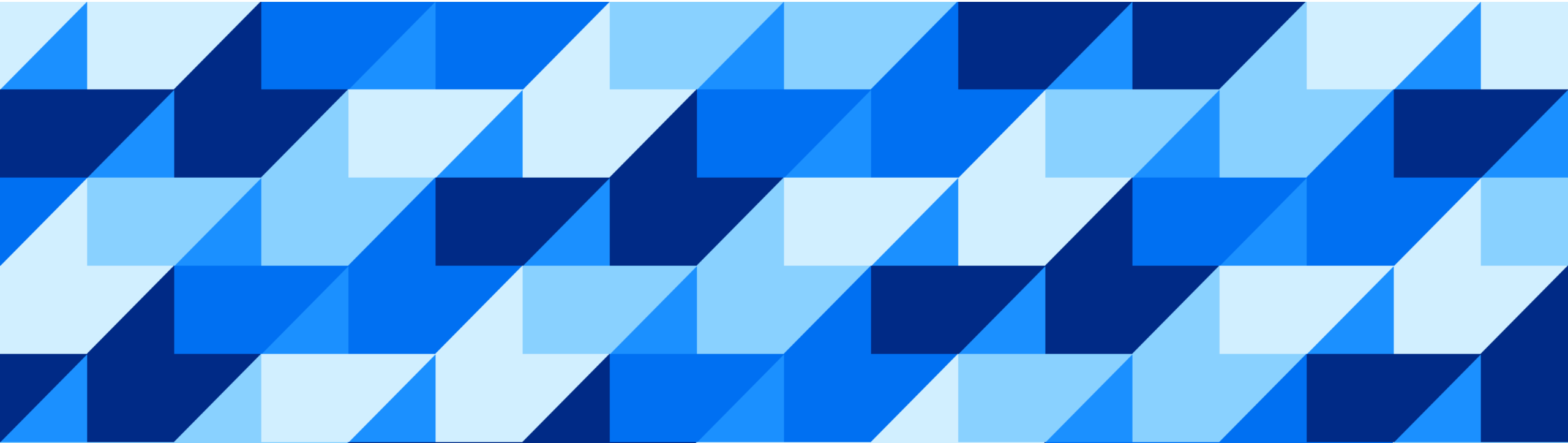
`Communication.IsEmailAddress`
/
`Communication.IsPhoneNumber`

`Communication.Address`

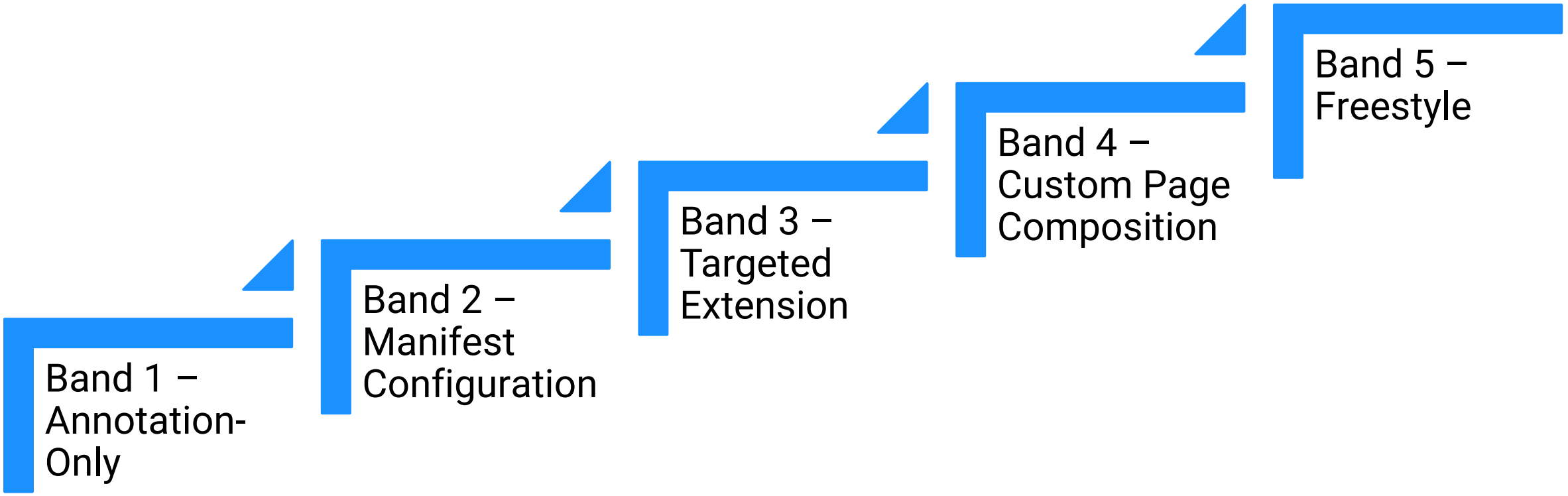
Where Annotations should Live



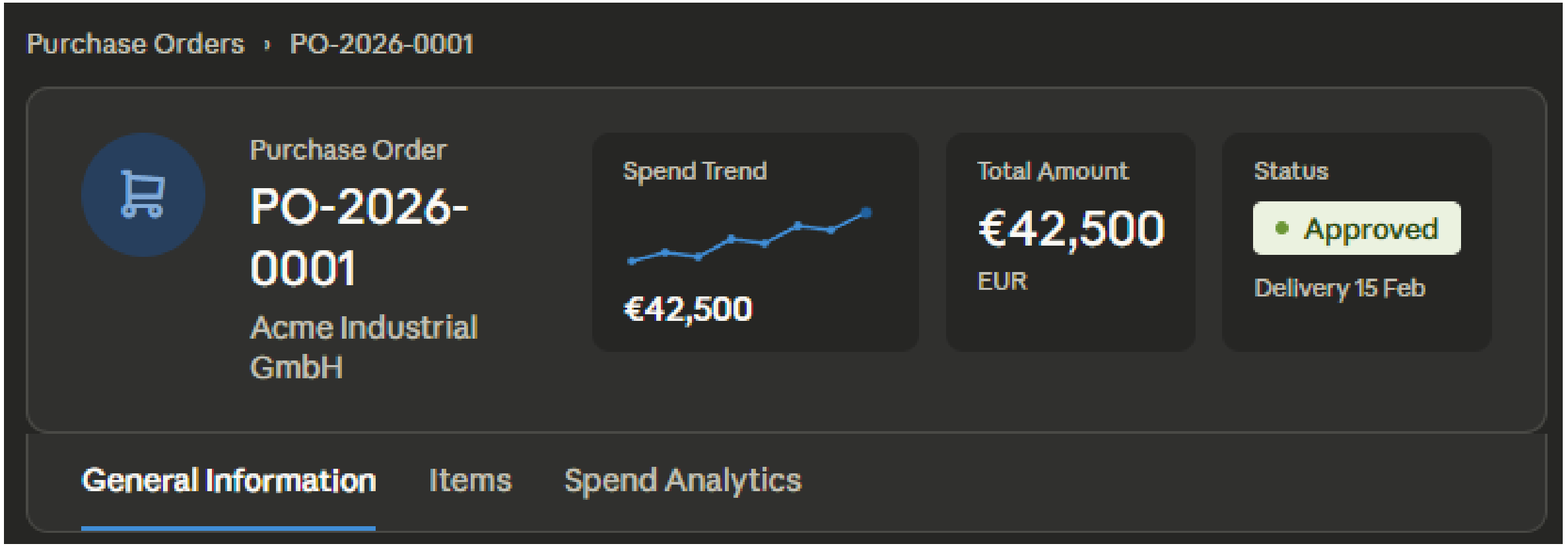
The Extension Spectrum



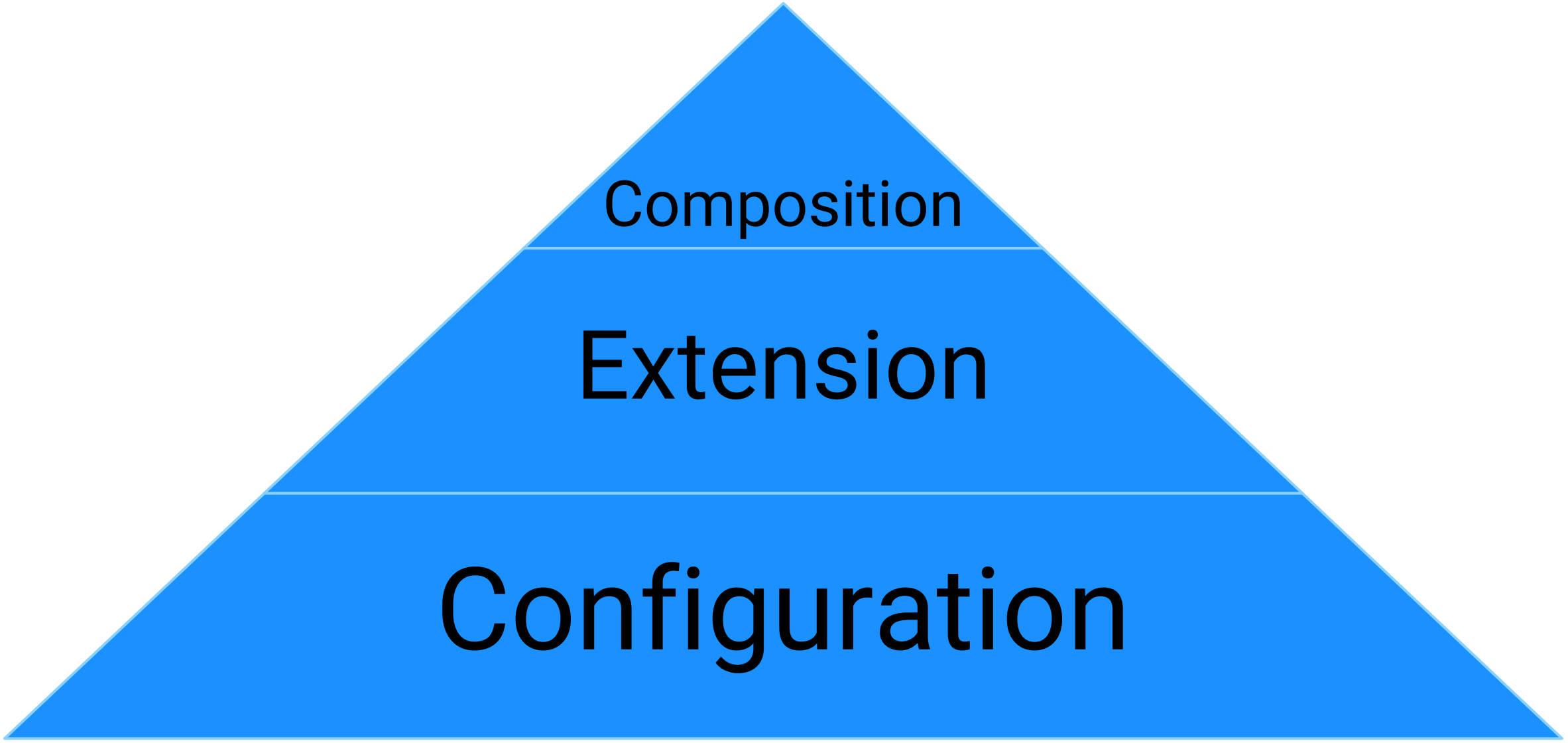
The Five Bands of Extension



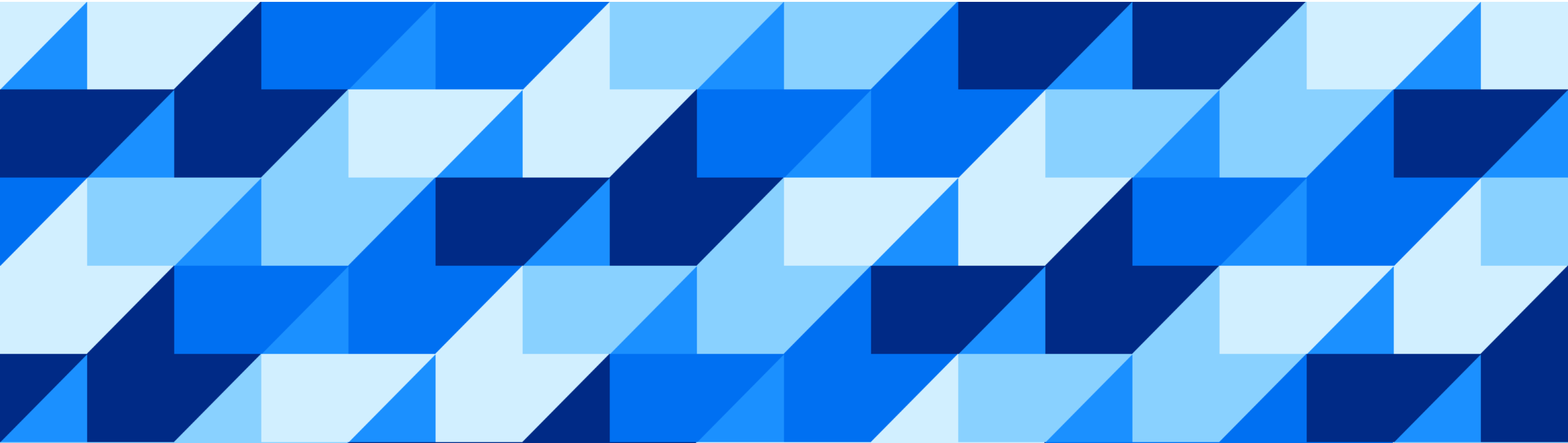
Example


Purchase Orders › PO-2026-0001
Purchase Order
PO-2026-0001
Acme Industrial GmbH
Spend Trend
€42,500
Total Amount
€42,500
EUR
Status
Approved
Delivery 15 Feb
General Information
Items
Spend Analytics

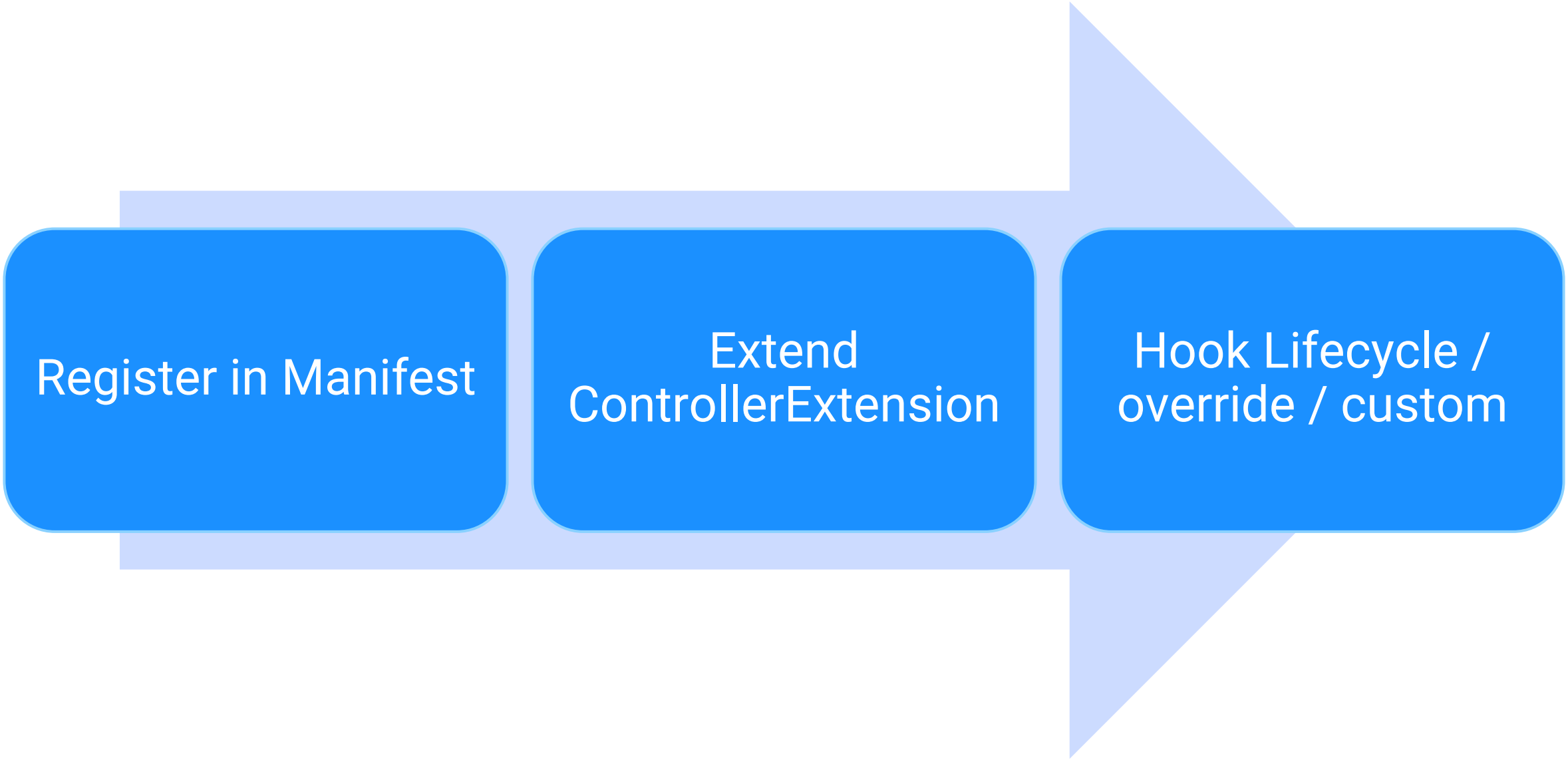
FPM — Three Layers, One Model



Extension Techniques



Controller Extensions



Controller Extensions – Registration in manifest.json example

```
"sap.ui5": {
  "extends": {
    "extensions": {
      "sap.ui.controllerExtensions": {
        "sap.fe.templates.ObjectPage.ObjectPageController": {
          "controllerName": "zrd.fiori.purchaseorders.ext.controller.ObjectPageExt"
        }
      }
    }
  }
}
```

Controller Extensions – Extension Example

```
sap.ui.define([
  "sap/ui/core/mvc/ControllerExtension"
], function (ControllerExtension) {
  "use strict";

  return
ControllerExtension.extend("zrd.fiori.purchaseorders.ext.controller.ObjectPageExt", {
    override: {
      onInit: function () {
        // local setup
      }
    },

    onMarkAsReviewed: function (oEvent) {
      var oContext = this.base.getExtensionAPI().getBindingContext();
      // call a service action, refresh, toast, etc.
    }
  });
});
```

Custom Actions – Buttons in Defined Places

Placement controlled by manifest: toolbar, header, footer, section

Three properties do most of the work:

`press`
`visible` / `enabled`
`requiresSelection`

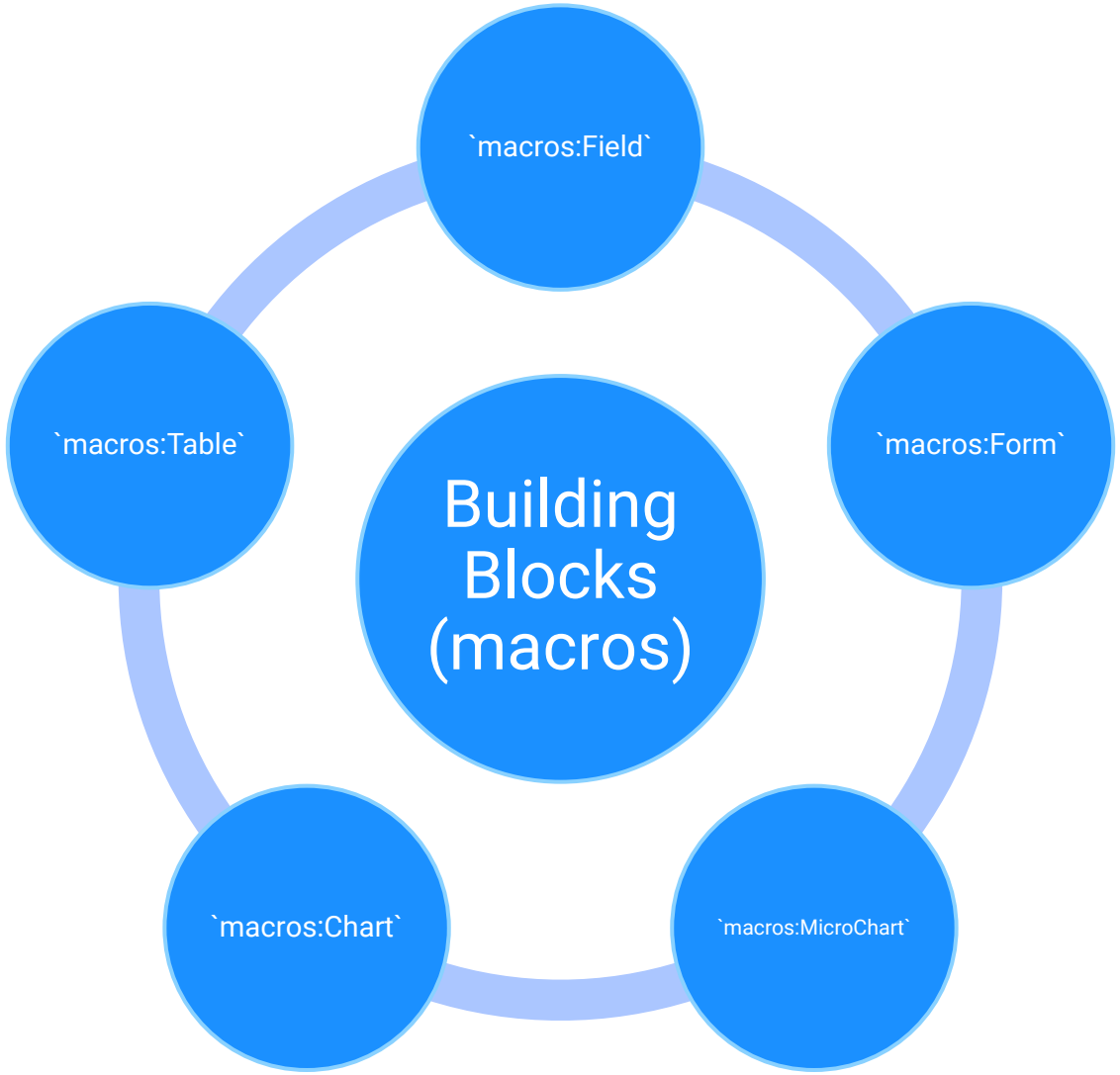
Custom Actions – Buttons in Defined Places - Example

```
"controlConfiguration": {
  "@com.sap.vocabularies.UI.v1.Identification": {
    "actions": {
      "MarkReviewed": {
        "press":
"zrd.fiori.purchaseorders.ext.controller.ObjectPageExt.onMarkAsReviewed",
        "visible": "{ui>/isEditable} === false",
        "enabled": "{= ${Status} !== 'Reviewed' }",
        "text": "Mark as Reviewed",
        "requiresSelection": false
      }
    }
  }
}
```

Custom Sections — Slot a Fragment Into the Object Page

```
"content": {
  "body": {
    "sections": {
      "SpendAnalytics": {
        "title": "Spend Analytics",
        "template": "zrd.fiori.purchaseorders.ext.view.SpendAnalytics",
        "position": {
          "placement": "After",
          "anchor": "GeneralInformationFacet"
        }
      }
    }
  }
}
```

Custom Sections — Slot a Fragment Into the Object Page



Custom Pages — Bespoke Layout, Standard Rendering

For layouts the standard floorplans cannot deliver (dashboards, workbenches, wizards)

Register as an FPM target:

`type`: Component

`name`: `sap.fe.core.fpm`

`viewName` + `contextPath` in settings

Inside the view: compose freely (Page, FlexBox, Grid, Splitter) and drop building blocks

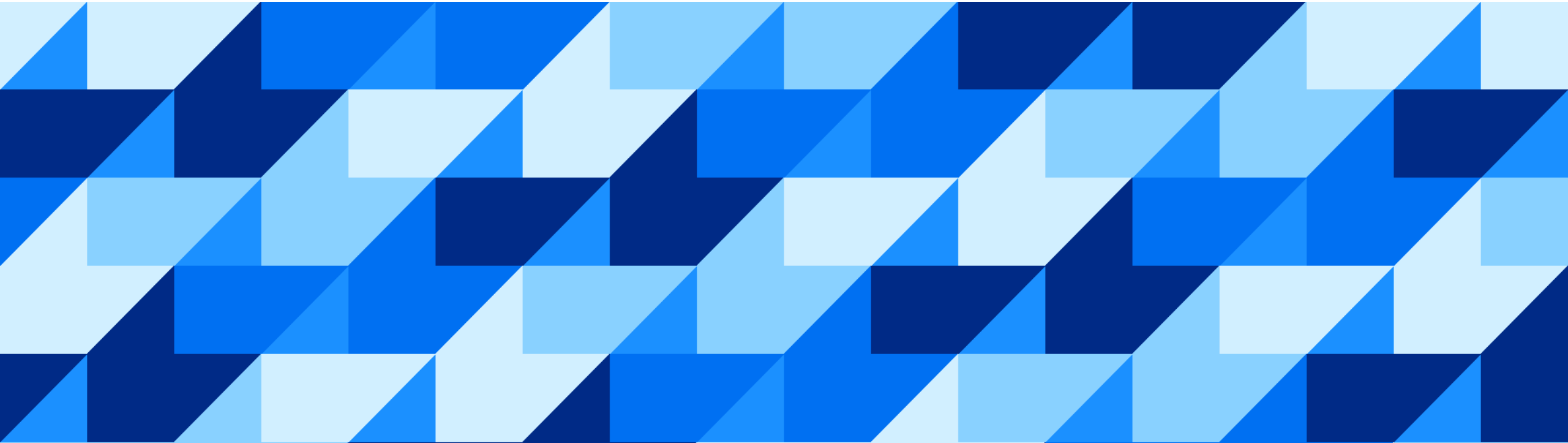
Custom Pages — Registration Example

```
"routing": {
  "routes": [
    {
      "pattern": "supplier-workbench/{supplierId}",
      "name": "SupplierWorkbench",
      "target": "SupplierWorkbench"
    }
  ],
  "targets": {
    "SupplierWorkbench": {
      "type": "Component",
      "id": "SupplierWorkbench",
      "name": "sap.fe.core.fpm",
      "options": {
        "settings": {
          "viewName": "zrd.fiori.purchaseorders.ext.view.SupplierWorkbench",
          "contextPath": "/Suppliers"
        }
      }
    }
  }
}
```

Custom Pages — View Example

```
<mvc:View
  xmlns:mvc="sap.ui.core.mvc"
  xmlns="sap.m"
  xmlns:macros="sap.fe.macros">
  <Page title="Supplier Workbench">
    <macros:FilterBar id="fb1"
metaPath="@com.sap.vocabularies.UI.v1.SelectionFields"/>
    <FlexBox direction="Row">
      <macros:Form id="frm1"
metaPath="@com.sap.vocabularies.UI.v1.FieldGroup#SupplierInfo"/>
      <macros:Chart id="chrt1"
metaPath="@com.sap.vocabularies.UI.v1.Chart#SpendByCategory"/>
    </FlexBox>
    <macros:Table id="tbl1" metaPath="@com.sap.vocabularies.UI.v1.LineItem#OpenPOs"/>
  </Page>
</mvc:View>
```

Adapations



Adaptation Projects vs Local Extensions

Local Extensions

You own the app

Code lives in `webapp/ext/`

Adaptation Project

SAP Delivers base app

You layer changes via a descriptor – base app never modified

What Adaptation Projects Can Do

Add, hide, or rename columns in List Report tables

Add or hide sections and subsections on Object Pages

Add custom actions and custom fragments

Override labels and texts

Adjust manifest properties (FCL, draft, navigation)

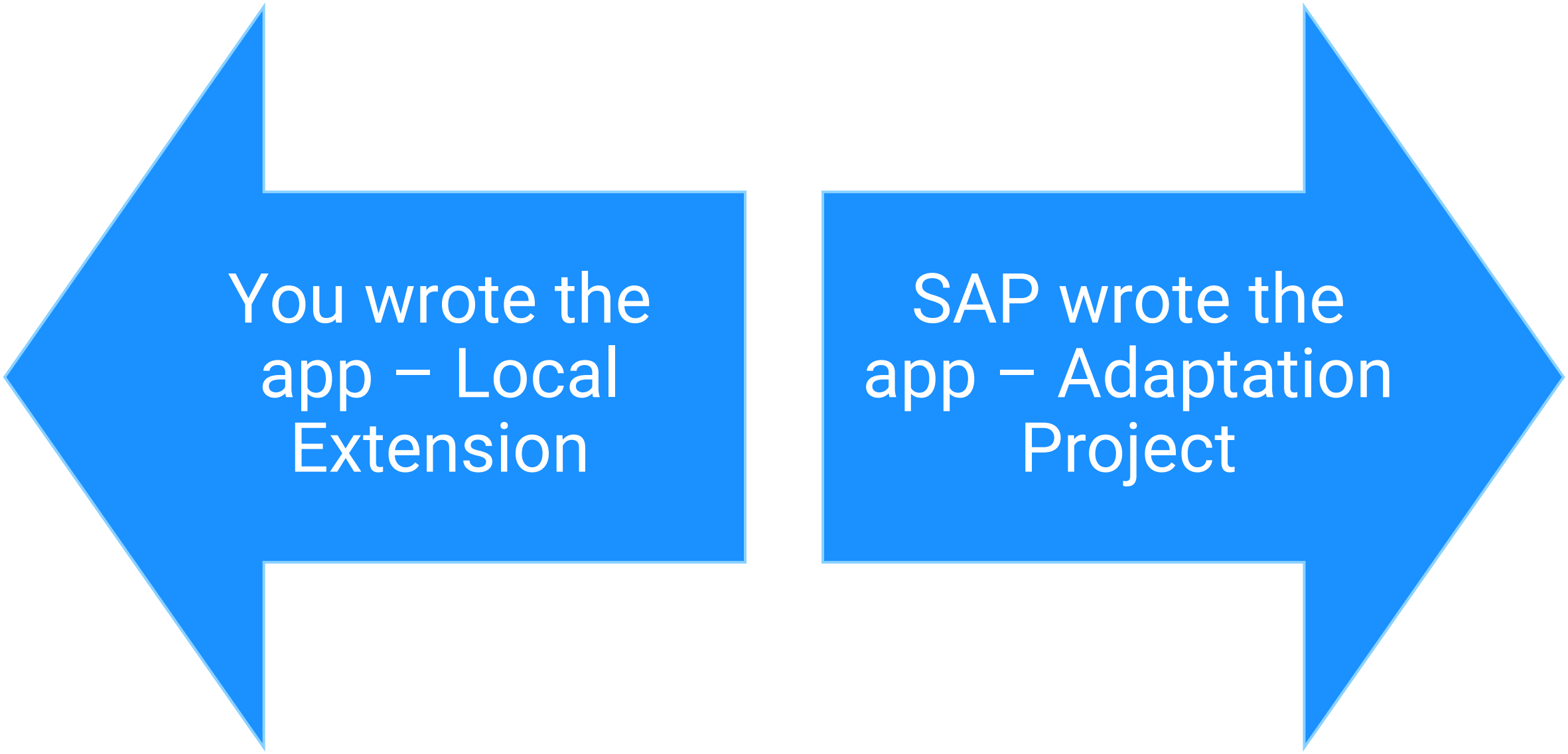
Where Adaptation Projects Live

A top-level project of its own — not part of the base app

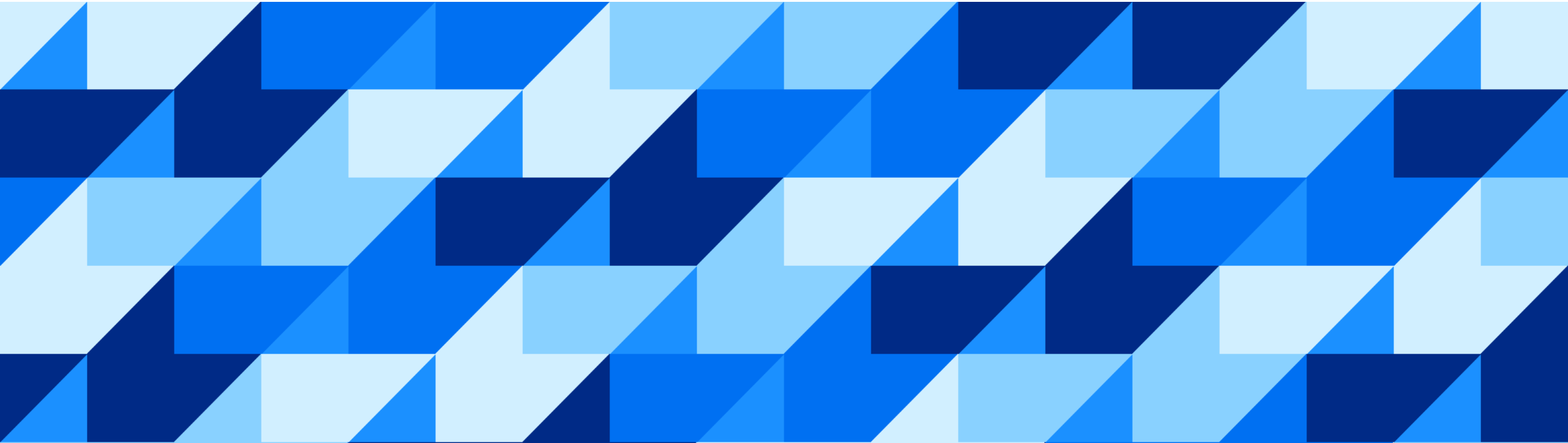
Template in SAP Build Code / BAS: SAPUI5 Adaptation Project

References the base app by its application descriptor (you point at it, you do not copy it)

When to use Which



Building for the Long Term



Anti-Patterns That Break Upgrades (1 of 2)

Accessing controls by generated ID	`this.byId("...")` against framework-generated IDs → use `extensionAPI` or your own IDs
Calling underscore-prefixed methods	Private contract, no upgrade guarantee → use documented `extensionAPI` only
`onAfterRendering` for cosmetic tweaks	DOM is internal → use Theme Designer, criticality annotations, or manifest `hidden` flags

Anti-Patterns That Break Upgrades (2 of 2)

Wrapping the whole page in freestyle	For one custom button — trust the framework; use Sections 6-8 patterns; for full layout control use a custom page
Business logic in the controller extension	Put it in a CAP / RAP service action; controller's job is UX (confirm, refresh, toast)
Sharing code through controller inheritance	Share via `ext/util/` modules; one controller extension per page

Testing Strategy — Five Layers

Annotation Preview	• Visual validation in the inner loop; mock-server-driven
Unit Tests	• For controller extensions — QUnit, mock `extensionAPI`
Integration Tests	• With OPA5 — full app, reusable page objects for LR / OP / custom pages
Backend contract tests	Mocha for CAP, ABAP Unit / RAP test classes for RAP
What NOT to test	• Framework behaviour, rendering details, generated control IDs

Performance Considerations

Request Batching	OData V4 batches by default; custom OData calls should join the batch
Lazy section loading	`loadingMode: 'lazyLoading'` for unlikely-scrolled sections
Draft handling costs	Don't enable drafts unless autosave / concurrency is genuinely needed
Annotation bloat	Large `LineItem` definitions slow first-load; audit periodically (usually 20-30% unused)
Custom Page composition cost	Many building blocks = many requests; consider tabs over single-page

Best Practices — Upgrade Safety & Project Hygiene

Upgrade Safety

- Use only the public `extensionAPI`
- Never access framework controls by generated ID
- Pin versions: `@sap/ui5-tooling`, `@sap/generator-fiori`, `@sap/cds`
- Run the generator's preview locally during development

Project Hygiene

- Extension code under `webapp/ext/` only — `controller/`, `view/`, `util/`
- Name extensions after the page (`ObjectPageExt.js`, `ListReportExt.js`)
- One controller extension per page; share via `util/` modules
- Annotations in dedicated files, not scattered across folders

Best Practices — Choosing the Right Band & AI-Assisted Development

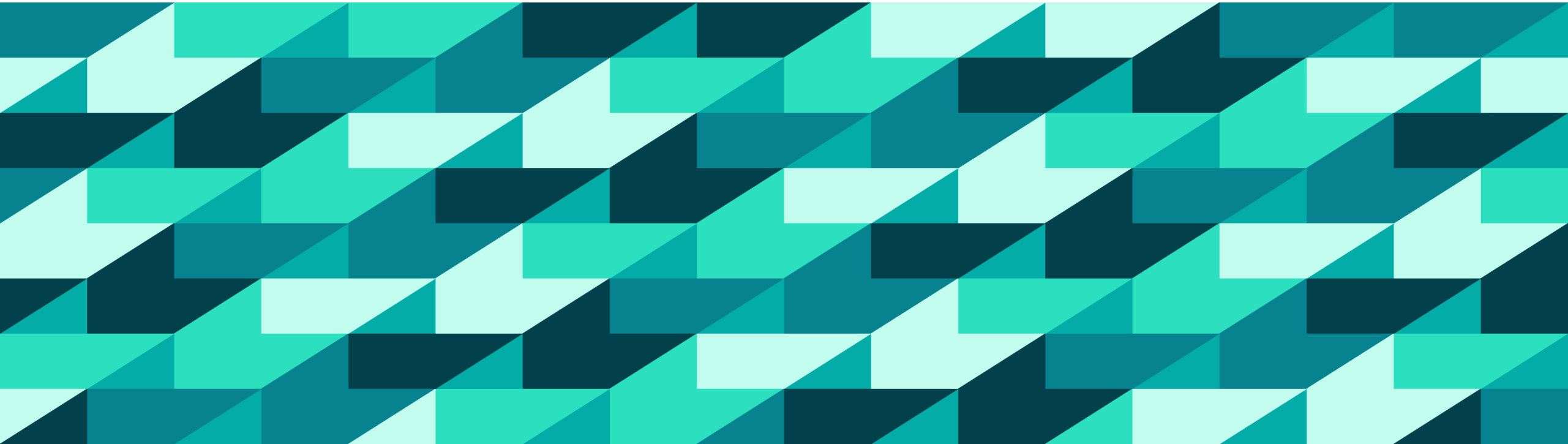
When a new requirement lands, ask the questions:

Annotations only?
Manifest only?
Defined extension point (action / section / column)?
Custom page with building blocks?
Freestyle?

AI-Assisted Development

SAP Build Code Joule integration

Demonstration



Conclusion

The shift in question

From *"should we extend Fiori Elements?"* to *"which band of extension fits this requirement?"*

The discipline going forward

Push every requirement as high up the bands as it will go

Reach for building blocks before reaching for code

Trust the extensionAPI — it is the contract that survives upgrades

Local extension and Adaptation Project are the same mental model with a different wrapper

Thank you.



SAP SE or an SAP affiliate company. All rights reserved. See Legal Notice on www.sap.com/legal-notice for use terms, disclaimers, disclosures, or restrictions related to this material.